

UseEffect advanced

details about it and more

source:

https://www.youtube.com/watch?v=QQYeipc_cik

TODO: <https://wanago.io/2022/04/11/abort-controller-race-conditions-react/>

First example

we have a button and a message. If we click on the button, the state will update...
BUT the <title> update has an delay...

```
const UserSwitching = () => {  
  //throttle internet to slow  
  const [user, setUser] = useState()  
  const [userId, setUserId] = useState(1)  
  useEffect(() => {  
    const controller = new AbortController()  
    const signal = controller.signal  
    fetch(`https://jsonplaceholder.typicode.com/users/${userId}`, { signal })  
      .then(res => res.json())  
      .then(data => setUser(data))  
  }, [userId])  
  function handleChange(e:any) {  
    setUserId(e.target.id)  
  }  
  return (  
    <div>  
      <div id="1" onClick={handleChange}>User 1</div>  
      <div id="2" onClick={handleChange}>User 2</div>  
      <div id="3" onClick={handleChange}>User 3</div>  
      <p>{JSON.stringify(user)}</p>  
    </div>  
  )  
}
```

Why is the delay:

you have to know, that we have three parties communicating with each other

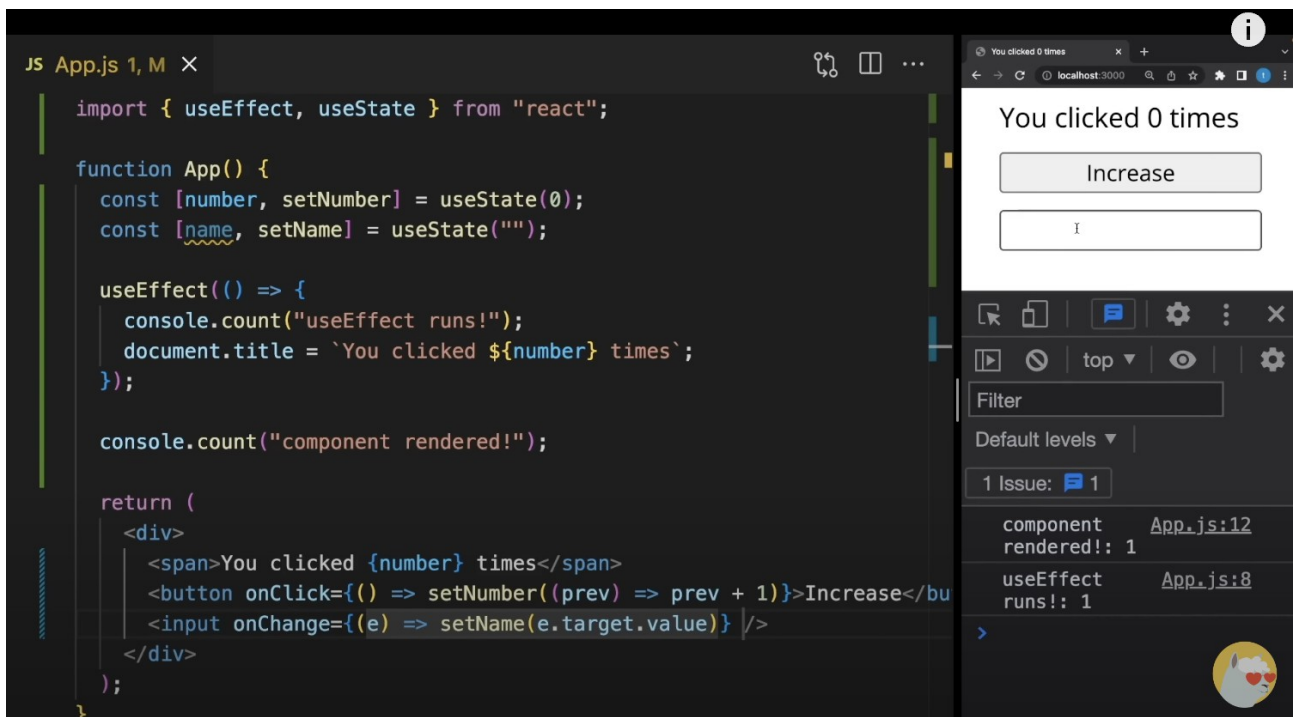
- Component
- React itself
- browser

component says to react, please update
after that, react tells the browser to update the <title>

But each of the updates above triggers a render.

Lets extend the example.

We will now have an input field to update a name state



```
JS App.js 1, M X
import { useEffect, useState } from "react";

function App() {
  const [number, setNumber] = useState(0);
  const [name, setName] = useState("");

  useEffect(() => {
    console.count("useEffect runs!");
    document.title = `You clicked ${number} times`;
  });

  console.count("component rendered!");

  return (
    <div>
      <span>You clicked {number} times</span>
      <button onClick={() => setNumber((prev) => prev + 1)}>Increase</button>
      <input onChange={(e) => setName(e.target.value)} />
    </div>
  );
}
```

You clicked 0 times

Increase

I

Filter

Default levels ▾

1 Issue: 1

component rendered!: 1 App.js:12

useEffect runs!: 1 App.js:8

every time I change the name input, the useEffect will run. But this shouldn't be happen, because the useEffect depends on the number.

This is the time when you have to use [dependencies] in useEffect.

The screenshot shows a code editor with the following JavaScript code:

```
import { useEffect, useState } from "react";

function App() {
  const [number, setNumber] = useState(0);
  const [name, setName] = useState("");

  useEffect(() => {
    console.count("useEffect runs!");
    document.title = `You clicked ${number} times`;
  }, [number]);

  console.count("component rendered!");

  return (
    <div>
      <span>You clicked {number} times</span>
      <button onClick={() => setNumber((prev) => prev + 1)}>Increase</button>
      <input onChange={(e) => setName(e.target.value)} />
    </div>
  );
}
```

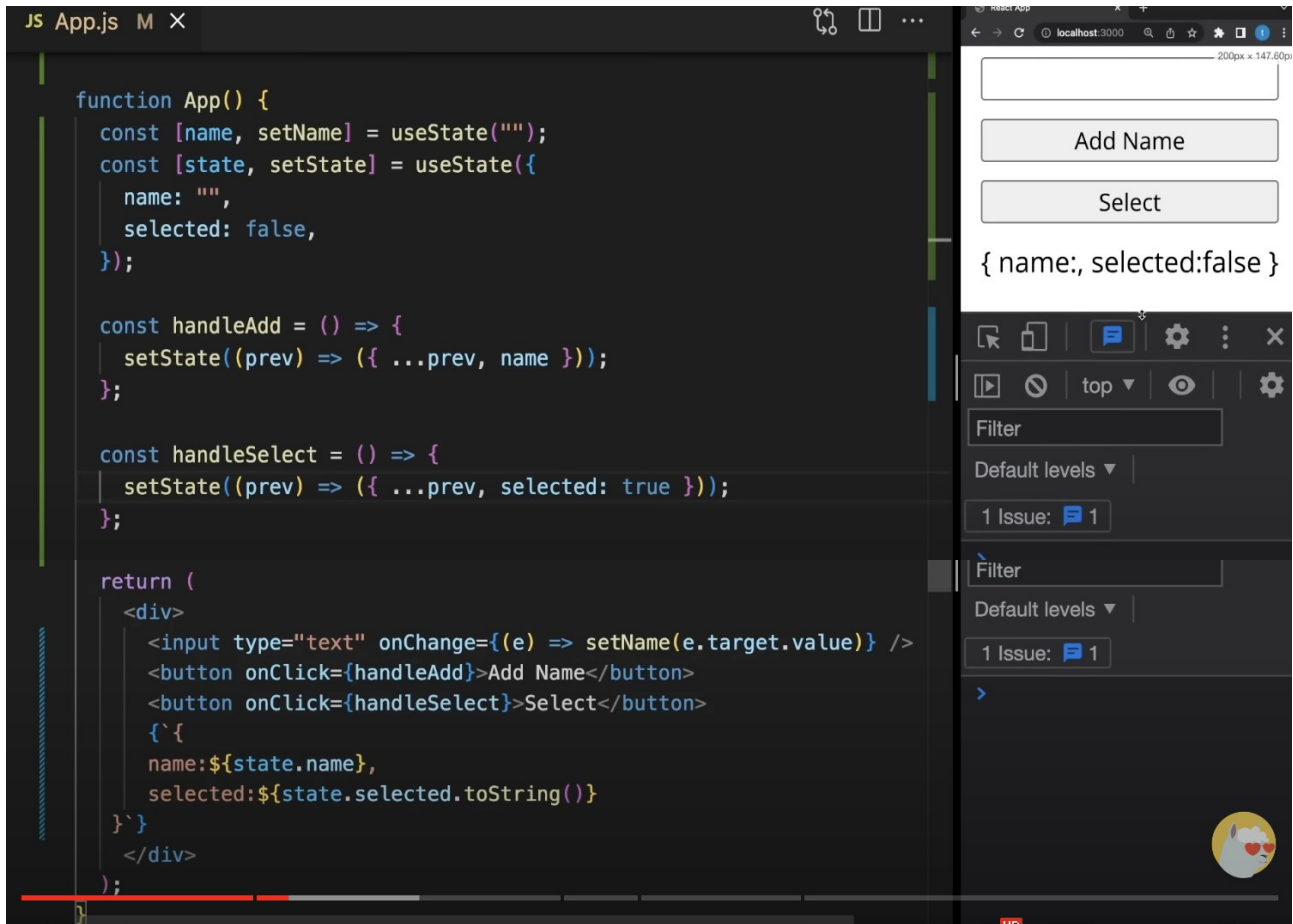
On the right, a browser window shows the application running at localhost:3000. The page title is "You clicked 0 times". There is an "Increase" button and an input field. Below the browser, a Chrome DevTools console shows the following log:

Message	File	Line
component rendered!: 1	App.js:12	
useEffect runs!: 1	App.js:8	

There is an important topic to understand. Working with successfully with useEffect [dependencies] has a lot to do with the type of value

- primitive
- non-primitive

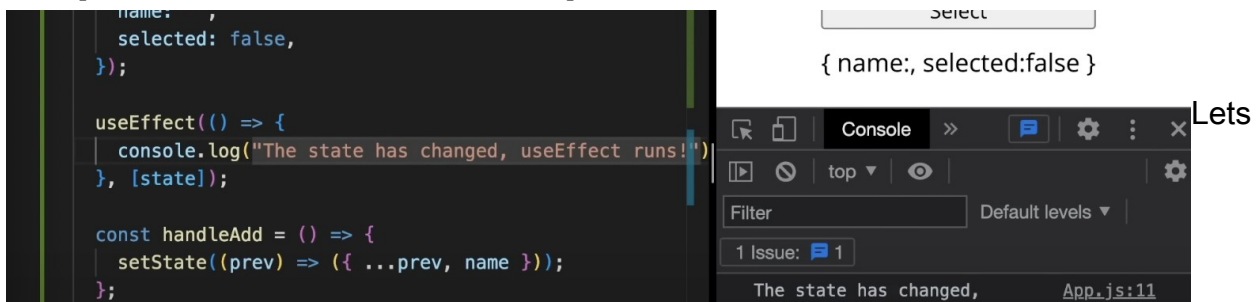
Second Example



We have now an state Object with two properties. {name: "", selected: false}

in the return we can update the properties...works fine...

lets put an useEffect to the component.



update both values – the useEffect is listening to the state Object and it runs after updating the state.name and state.selected.



BUT if I click on the button to update the state.selected – which is again a false value – the useEffect runs again. But the value didn't change in the client output. Same for the name „john“.

WHY IS THAT?

This is related to the primitive and non-primitive values...

Explain primitive and non-primitives.

String, numbers, null, undefined, booleans are primitives

```
1 === 1 => true
true === true => true
false === false => true
„1“ === „1“ => true
```

objects and arrays are non-primitive values.

```
Const x = {name: „john“}
const y = {name: „john“}
```

```
x === y => false
```

non-primitive values referring to different memory spots. The x and y look the same, but they have different spots in the memory..meaning...they are not equal.

They are 2 different objects

if I refer x = y and I run

```
x === y => true
```

SAME WITH OBJECTS.

In useEffect we can solve this issue in different ways.

UseMemo solution.

```
const user = useMemo(
  () => ({
    name: state.name,
    selected: state.selected,
  }),
  [state.name, state.selected]
);
```

1 Issue: 1

> [] === []

< false

> [1] === [1]

< false

> 1===1

< true

Then change the useEffect dependence

```
useEffect(() => {
  console.log("The state has changed, useEffect runs!");
}, [user]);
```

Select

{ name:, selected:false }

OR

Another way is to write each dependency single by single...

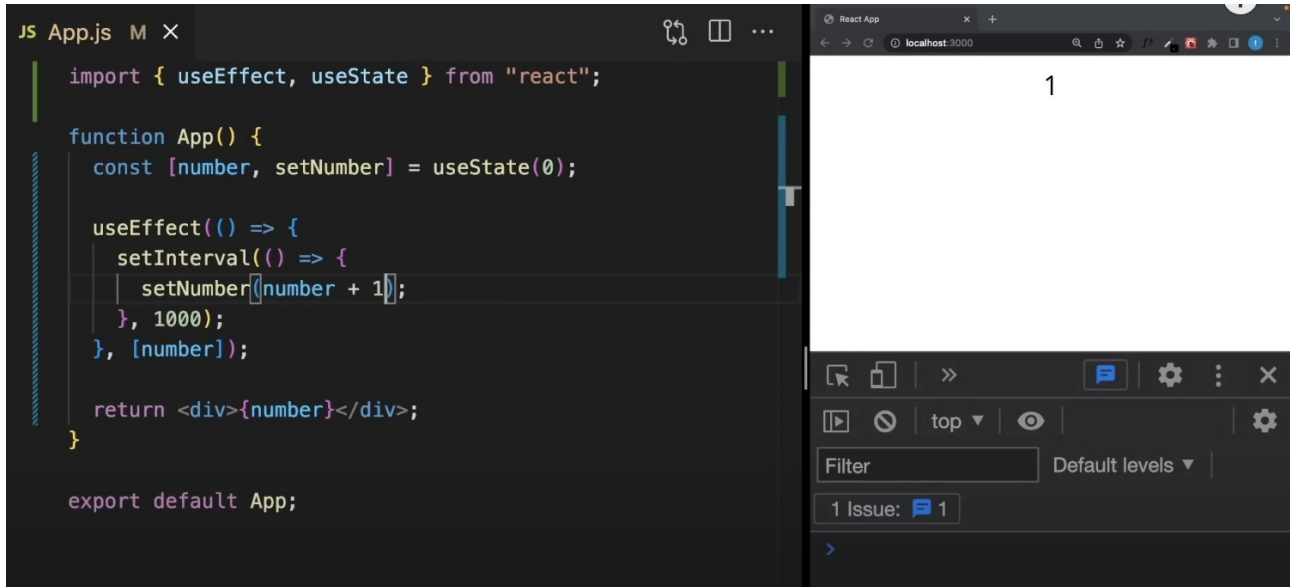
```
useEffect(() => {
  console.log("The state has changed, useEffect runs!");
}, [state.name, state.selected]);
```

{ name:, selected:false }

this is not convenient

both solutions will make the useEffect behavior correct.

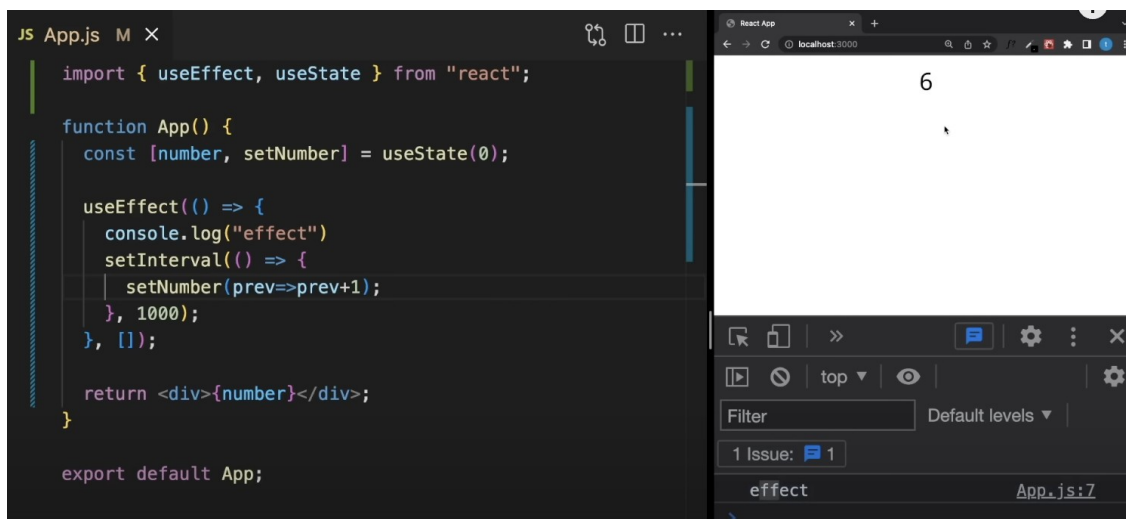
Update the state correct



This component will update every 1000ms the number by 1

If you have a closer look, you will see a kind of glitch at the number...
you will have now an infinite loop.

To avoid the infinite loop, you have to use an updating function to the number



useEffect CleanUp functions

Taking the above intervall example, we added some values to the return manually.

The screenshot shows a code editor with the following JavaScript code in `App.js`:

```
import { useEffect, useState } from "react";

function App() {
  const [number, setNumber] = useState(0);

  useEffect(() => {
    console.log("effect")
    setInterval(() => {
      setNumber(prev=>prev+1);
    }, 1000);
  }, []);

  return <div>{number}asdsadsa2e</div>;
}

export default App;
```

The browser window on the right shows the output: `61asdsadsa2e`. The React DevTools component inspector on the bottom right shows a single issue: `effect` at `App.js:7`.

You will see, the output is kind of messed up. The reason for that is, that with each typing of characters we are starting a new intervall.

To clean that up, we will use a cleanup function.

=> **new Example for demo CLEANUP :**

The screenshot shows a code editor with the following JavaScript code in `App.js`:

```
import { useEffect, useState } from "react";

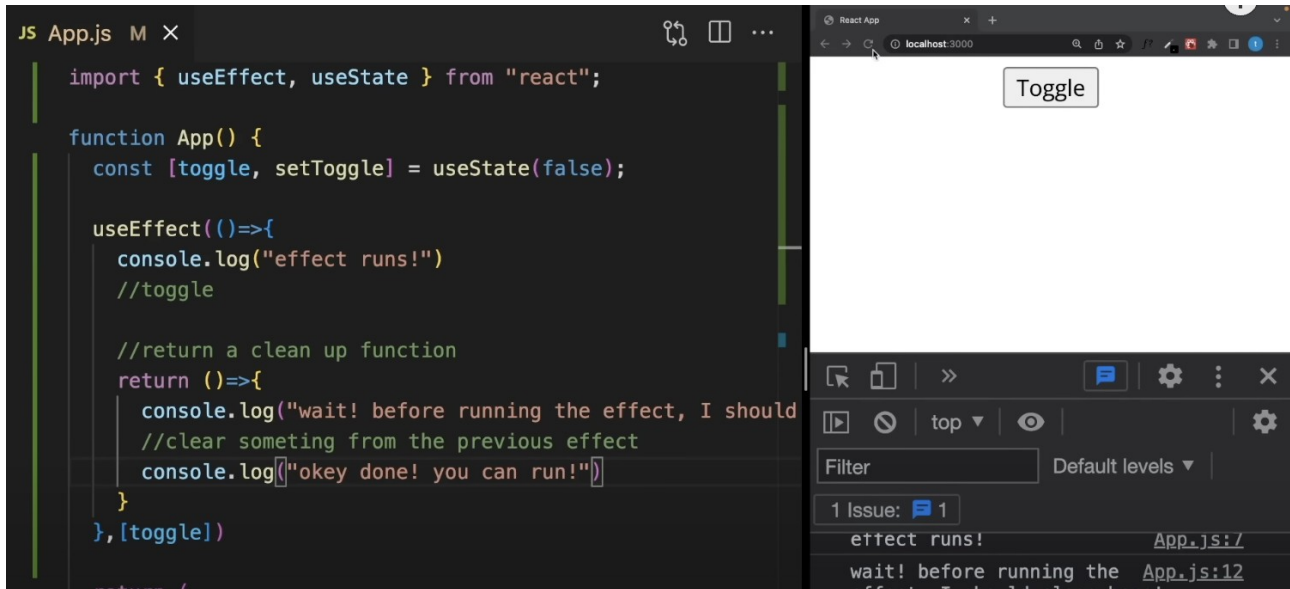
function App() {
  const [toggle, setToggle] = useState(false);

  useEffect(()=>{
    console.log("effect runs!")
    //return a clean up function
  },[toggle])

  return (
    <div>
      <button onClick={() => setToggle(!toggle)}>Toggle</button>
    </div>
  );
}
```

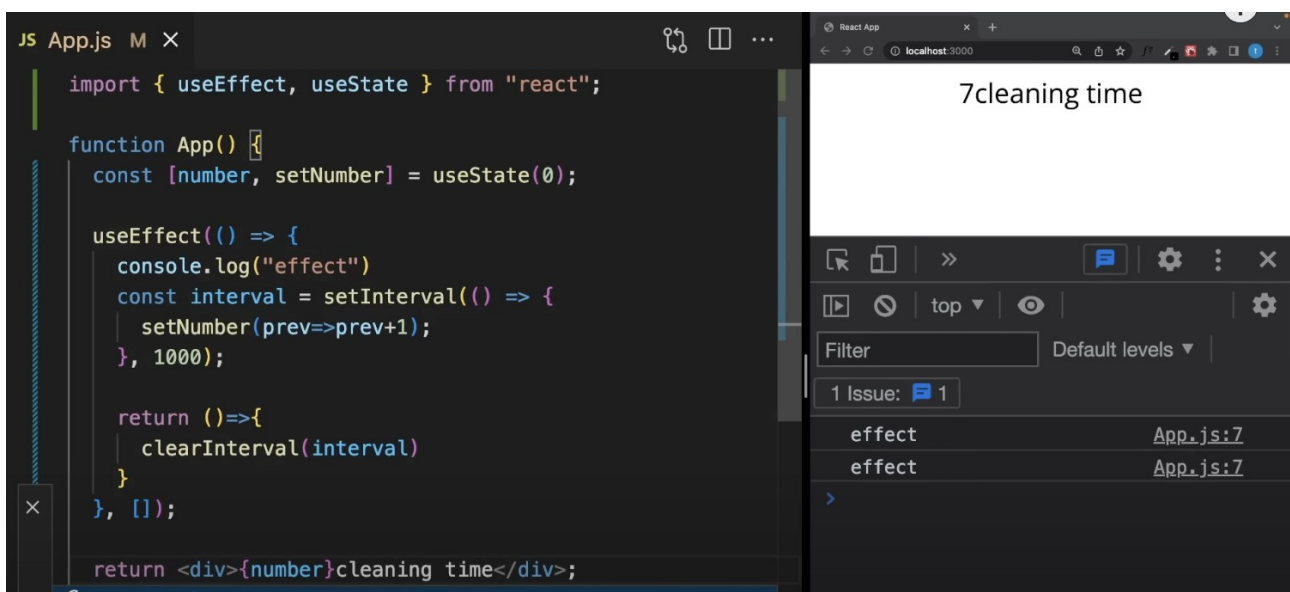
The browser window on the right shows the output: a button labeled `Toggle`. The React DevTools component inspector on the bottom right shows a single issue: `effect runs!` at `App.js:7`.

With each toggle click, the useEffect will run.

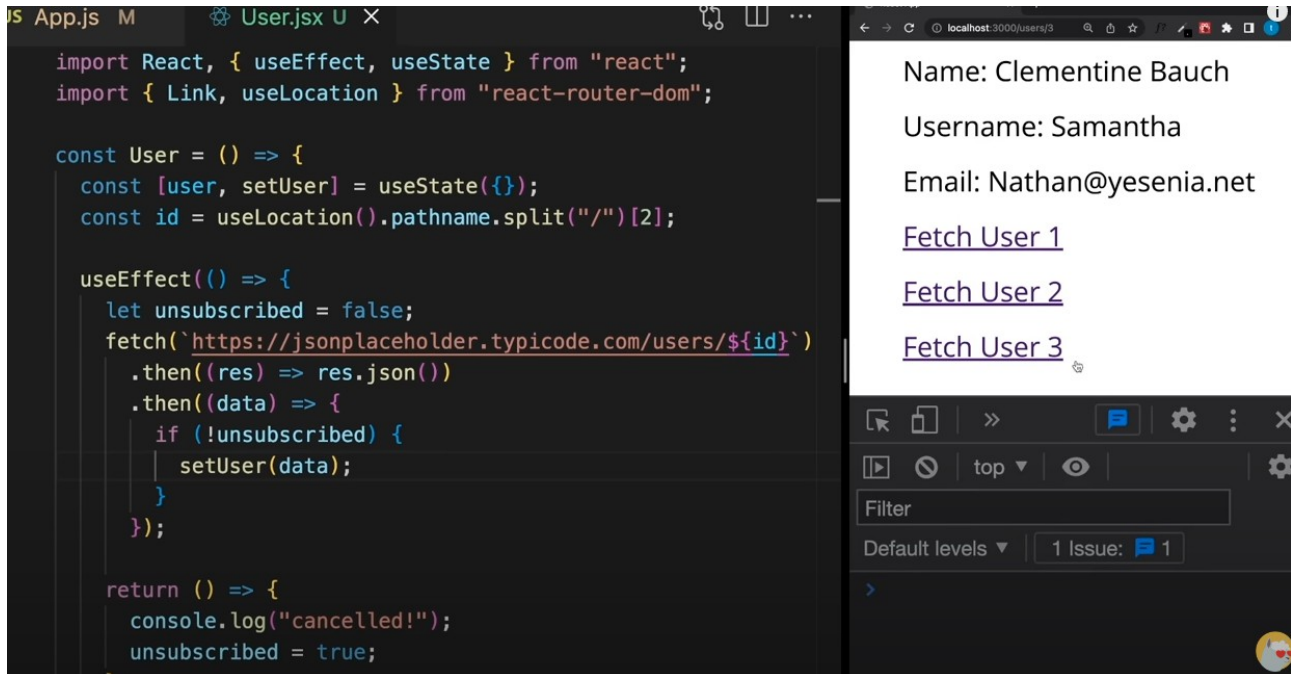


Lets add an cleanup function to demo the behavior

Going back to the interval example...we will add the cleanup function to it



Example user fetching



with this example there will be only one user shown.

More professional solution with AbortController()

https://youtu.be/QQYeipc_cik?t=1058

```
const User = () => {
  const [user, setUser] = useState({});
  const id = useLocation().pathname.split("/") [2];

  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`)
      .then((res) => res.json())
      .then((data) => {
        setUser(data);
      });
  }, [id]);

  return (
    <div>
      <p>Name: {user.name}</p>
      <p>Username: {user.username}</p>
      <p>Email: {user.email}</p>
      <Link to="/users/1">Fetch User 1</Link>
      <Link to="/users/2">Fetch User 2</Link>
      <Link to="/users/3">Fetch User 3</Link>
    </div>
  );
}
```

Name: Leanne Graham

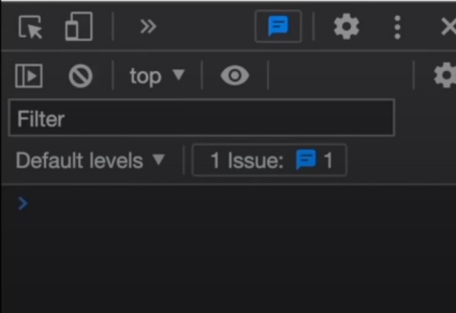
Username: Bret

Email: Sincere@april.biz

[Fetch User 1](#)

[Fetch User 2](#)

[Fetch User 3](#)



Now if you click quick from user one to user two, you will see a big delay.. this is because of the not stopped (- cleaned) request... SOLUTION

Cleanup your request

```
useEffect(() => {
  const controller = new AbortController();
  const signal = controller.signal;

  fetch(`https://jsonplaceholder.typicode.com/users/${id}`, {
    .then((res) => res.json())
    .then((data) => {
      setUser(data);
    })
  })

  return () => {
    console.log("cancelled!")
    controller.abort();
  };
}, [id]);

return (
  <div>
    <p>Name: {user.name}</p>
    <p>Username: {user.username}</p>
```

Name: Clementine Bauch

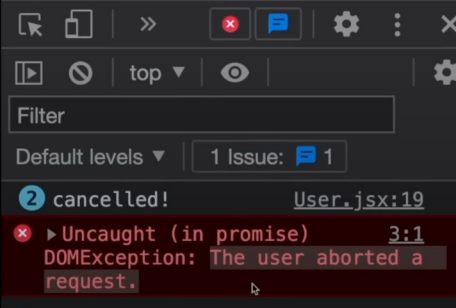
Username: Samantha

Email: Nathan@yesenia.net

[Fetch User 1](#)

[Fetch User 2](#)

[Fetch User 3](#)



Now if you click quick, the data is precise